

Compresión de Imágenes mediante la DCT (Discrete Cosine Transform)

Tyrone Miguel Novillo Bravo¹, Freddy Eduardo López Padilla²

Facultad de Ingeniería, Universidad de Cuenca, Cuenca, Azuay, Ecuador¹

Facultad de Ingeniería, Universidad de Cuenca, Cuenca, Azuay, Ecuador²

Resumen This paper presents the design and implementation of a graphical application for image compression based on the two-dimensional Discrete Cosine Transform (DCT). The system allows users to load a color image or capture one via webcam, convert it to grayscale, and compress it by retaining only a user-defined number of DCT coefficients selected through zigzag scanning of 8×8 pixel blocks. The compression quality is evaluated using the Peak Signal-to-Noise Ratio (PSNR).

Keywords: DCT, PSNR, Zigzag

1. Introducción

La herramienta matemática, base del estándar JPEG, es la Transformada Discreta del Coseno (DCT). La DCT descompone bloques de píxeles en componentes frecuenciales y concentra la mayor parte de la energía de la imagen en un reducido número de coeficientes de baja frecuencia, propiedad conocida como compactación de energía. Esta característica permite descartar los coeficientes de alta frecuencia con un impacto visual mínimo, obteniendo una representación compacta de la imagen. La selección de qué coeficientes conservar se realiza mediante el recorrido en zigzag sobre cada bloque de 8×8 coeficientes. Este barrido diagonal prioriza las frecuencias bajas (mayor energía) y desplaza las altas (menor energía) hacia el final del recorrido, de modo que al truncar la secuencia en k elementos siempre se retiene la información más significativa del bloque.

Para cuantificar el impacto de la compresión sobre la calidad de la imagen reconstruida, se emplea el PSNR (Peak Signal-to-Noise Ratio), una métrica expresada en decibeles que relaciona el error cuadrático medio entre la imagen original y la reconstruida con el nivel máximo de señal posible.

La presente práctica tiene como objetivo implementar un sistema completo de compresión y evaluación de imágenes basado en la DCT, con una interfaz gráfica que permite cargar imágenes a color o capturarlas con la cámara, convertirlas a escala de grises, comprimirlas especificando el número de coeficientes k mediante selección zigzag, visualizar la imagen reconstruida y reportar el PSNR resultante. Se realizan experimentos con cuatro imágenes distintas, dos capturadas con cámara y dos previamente almacenadas, variando el parámetro k para analizar la relación entre tasa de compresión y calidad visual.

2. Marco teórico

2.1. Representación digital de imágenes

Una imagen se considera una función bidimensional, donde cada par de coordenadas corresponde a un píxel cuyo valor representa la intensidad de luminosidad de ese punto. En imágenes a color, cada píxel almacena tres canales: rojo (R: Red), verde (G: Green) y azul (B: Blue), en el orden: (R, G, B), con valores enteros en el rango de 0 a 255, lo que corresponde codificar cada canal con un total de 8 bits. Para procesamiento mediante la DCT, la imagen se convierte previamente a escala de grises, obteniendo una única matriz de valores de luminancia, lo que reduce la dimensionalidad del problema sin alterar el contenido estructural de la imagen [2].

2.2. Transformada Discreta del Coseno

La Transformada Discreta del Coseno (DCT: Discrete Cosine Transform) es una herramienta matemática que descompone una señal discreta en una suma ponderada de funciones coseno de distintas frecuencias. A diferencia de la Transformada de Fourier, la DCT trabaja exclusivamente con números reales y concentra la mayor parte de la energía de la señal en un pequeño número de coeficientes de baja frecuencia, propiedad conocida como compactación de energía. DCT realiza redundancia espacial de la imagen a través de la compactación de energía en el dominio frecuencial. En la DCT Bidimensional (DCT 2D), el coeficiente (0,0) recibe el nombre de componente DC y representa el valor promedio del bloque; los demás se denominan componentes AC y codifican las variaciones de alta frecuencia (detalles y bordes). Por esta razón, la DCT es el núcleo del estándar de compresión JPEG y de numerosas aplicaciones de codificación multimedia [1].

2.3. Compresión por Bloques 8x8 y Recorrido zigzag

En el esquema de compresión adoptado, la imagen monocromática se divide en bloques no solapados de 8×8 píxeles. A cada bloque se le aplica la DCT 2D de forma separable: primero sobre columnas (dirección vertical) y luego sobre filas (dirección horizontal). Esto produce una matriz de 64 coeficientes por bloque, donde los de mayor magnitud se agrupan en la esquina superior izquierda (bajas frecuencias) [1].

Para explotar esta distribución, los 64 coeficientes se recorren en orden zigzag: un barrido diagonal que comienza en la componente DC (0,0) y avanza progresivamente hacia las frecuencias más altas, terminando en la posición (7,7). Este orden garantiza que los coeficientes de mayor energía se seleccionen primero. La compresión se logra conservando únicamente los primeros k coeficientes del recorrido y asignando cero a los restantes; a mayor k , mayor fidelidad pero menor compresión [4].

2.4. Métricas de Calidad: PSNR

La calidad de la imagen reconstruida se cuantifica mediante, primero, el Error Cuadrático Medio (MSE), que mide la diferencia promedio al cuadrado entre los píxeles originales y los reconstruidos. Su fórmula se muestra a continuación:

$$MSE = \frac{1}{M \cdot N} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \|I(i, j) - K(i, j)\|^2 \quad (1)$$

En la Ecuación 1, $I(i, j)$ representa la imagen original y $K(i, j)$ la reconstruida.

$M \cdot N$ es el número total de píxeles, por ello se divide para el total, obteniendo el promedio.

El PSNR (Peak Signal-to-Noise Ratio) transforma el MSE a escala logarítmica en decibeles (dB), facilitando la interpretación perceptual:

$$PSNR_{dB} = 10 \log_{10} \frac{(2^n - 1)^2}{MSE} \quad (2)$$

Donde n es el número de bits por muestra en la imagen, como es un píxel de 8 bits, el valor 255 correspondería a su nivel máximo.

Un PSNR superior a 40 dB indica una calidad excelente con diferencias imperceptibles al ojo humano; valores entre 30 y 40 dB representan calidad aceptable con leve degradación; por debajo de 30 dB los artefactos de bloque propios de la compresión DCT comienzan a ser visibles. Cuando $k = 64$ (todos los coeficientes conservados), el MSE es nulo y el PSNR tiende a infinito, indicando reconstrucción perfecta [3].

3. Desarrollo de la Práctica

3.1. Compresión de imagen monocromática

Para esta parte se utilizan dos funciones del código: *_zigzag_indices* y *compress_dct*.

```
def _zigzag_indices(n: int = 8):
    indices = []
    r, c = 0, 0
    going_up = True
```

Esta primera función genera el orden de recorrido zigzag para una matriz de $n \times n$, siendo en este caso $n=8$, por lo que se trabajará con bloques de 8×8 píxeles. La primera línea dentro de esta función es una lista vacía que almacenará las 64 posiciones (fila, columna) en orden zigzag. La posición inicial es la fila $r=0$ y la columna $c=0$, la cual es la esquina superior izquierda, que corresponde al coeficiente DC.

La última línea es una variable booleana *e* indica la dirección de la diagonal. Si es verdadera, la dirección de la diagonal será hacia arriba y a la derecha (ascendente). Si es falsa, será hacia abajo y a la izquierda (descendente). Esto se alterna entre cada rebote con los bordes.

```

if going_up:
    if r == 0 and c < n - 1:
        c += 1
        going_up = False
    elif c == n - 1:
        r += 1
        going_up = False
    else:
        r -= 1
        c += 1

```

Si dicha variable es verdadera, existen tres casos diferentes a tratar. En el primero, la posición es en el borde superior, y aún no se llega al borde derecho, por ende se debe avanzar una columna a la derecha y cambiar la dirección de la diagonal a descendente. El segundo caso ocurre en el borde derecho, por tanto, ahora se baja una fila, y se cambia la dirección. El último caso pertenece al comportamiento normal de la diagonal ascendente, donde se sube una fila y se avanza una columna.

```

else:
    if c == 0 and r < n - 1:
        r += 1
        going_up = True
    elif r == n - 1:
        c += 1
        going_up = True
    else:
        r += 1
        c -= 1

```

Para la diagonal descendente también existen tres casos, y son equivalentes a los anteriores. El primer caso ocurre al borde izquierdo, y si aún no se llega al borde inferior, entonces se baja una fila y se cambia la dirección a diagonal ascendente. Cuando ya se llega al borde inferior, se avanza una columna, y luego se cambia la dirección. El último caso corresponde a la diagonal en descendente. La función *_zigzag_indices* devuelve la lista completa de 64 posiciones en orden zigzag: *indices*, que luego es almacenada en la variable: *ZZ*.

La segunda función *compress_dct* recibe la imagen en escala de grises como matriz NumPy (*img_gray*) y el número de coeficientes a conservar (*num_coef*). De esta imagen se extrae su alto (*h*) y ancho (*w*) en píxeles.

Luego, se convierten los píxeles de enteros de 8 bits (*uint8*) a decimales de 64 bits (*float64*), puesto que la DCT produce valores decimales negativos que no se

pueden representar con *uint8*.

```
ph = int(np.ceil(h / 8) * 8)
pw = int(np.ceil(w / 8) * 8)
padded = np.pad(data, ((0, ph - h), (0, pw - w)), mode="edge")
result = np.zeros_like(padded)
```

Esta sección del código agrega filas y columnas extra al borde derecho e inferior usando *mode="edge"*, que repite el valor del píxel del borde (padding). La última línea crea una matriz de ceros del mismo tamaño que la matriz *padded*; aquí se irán escribiendo los bloques 8x8 reconstruidos uno a uno.

```
for bi in range(0, ph, 8):
    for bj in range(0, pw, 8):
        block = padded[bi : bi + 8, bj : bj + 8]
        coefs = dct(dct(block, norm="ortho", axis=0), norm="ortho",
                    , axis=1)
```

El siguiente código itera sobre las filas (*bi*) y las columnas (*bj*) del bloque actual de 8x8 píxeles.

Dentro de estas mismas líneas se aplica la DCT 2D de forma separable en dos pasos, con *axis=0* se aplica la DCT a cada columna (dirección vertical), y con *axis=1* se aplica a cada fila del resultado anterior (dirección horizontal). El parámetro *norm="ortho"* evita escalados incorrectos y permite la normalización de la transformada.

```
mask = np.zeros((8, 8), dtype=np.float64)
for k in range(num_coef):
    r, c = ZZ[k]
    mask[r, c] = coefs[r, c]
    recon = idct(idct(mask, norm="ortho", axis=1), norm="ortho",
                , axis=0)
    result[bi : bi + 8, bj : bj + 8] = recon

cropped = result[:h, :w]
return np.clip(np.round(cropped), 0, 255).astype(np.uint8)
```

Ahora, para reconstruir con IDCT 2D se utiliza esta sección del código, donde primero se crea una nueva matriz 8x8, que almacenará únicamente los coeficientes deseados.

Dentro del bucle for, que solo itera hasta el número de coeficientes a conservar, se obtiene la posición (fila, columna) del k-ésimo coeficiente según el orden zigzag, y luego copia el valor del coeficiente original a la nueva matriz *mask*. Todo lo que no se copia queda en 0, que sería información descartada. Después, con la IDCT sobre filas (*axis=1*) y luego sobre columnas (*axis=0*), se reconstruye el bloque de píxeles a partir de los coeficientes conservados, pero en la línea siguiente a

esta se escribe el bloque reconstruido en la posición correcta de la matriz *result*. Al finalizar todas estas iteraciones, se elimina el padding que se agregó al inicio, y se recorta de vuelta al tamaño original.

Esta función retorna una matriz con píxeles enteros, valores en el rango de 0 a 255 y sin signo.

3.2. Cálculo del PSNR de la imagen comprimida

```
def psnr(original: np.ndarray, reconstructed: np.ndarray) -> float:
    mse = float(
        np.mean((original.astype(np.float64) - reconstructed.astype(np
            .float64)) ** 2)
    )
    if mse < 1e-10:
        return float("inf")
    return float(20 * np.log10(255.0 / math.sqrt(mse)))
```

En esta función, se reciben dos matrices NumPy del mismo tamaño, *original*, la imagen monocromática antes de comprimir, y *reconstructed*, la imagen reconstruida. Devuelve un valor flotante en decibels.

Dentro de la variable *mse* se realiza el mismo procedimiento que en la Ecuación 1, se restan las dos matrices elemento por elemento, se eleva cada diferencia al cuadrado y se calcula el promedio. Finalmente, se convierte a flotante para ser usada en la Ecuación 2, que se encuentra en la última línea del código mostrado. Existe un caso especial en que, si el MSE es menor a 1×10^{-10} significa que la imagen reconstruida es idéntica a la original, por tanto se devuelve *float("inf")* que es infinito.

3.3. Experimentación

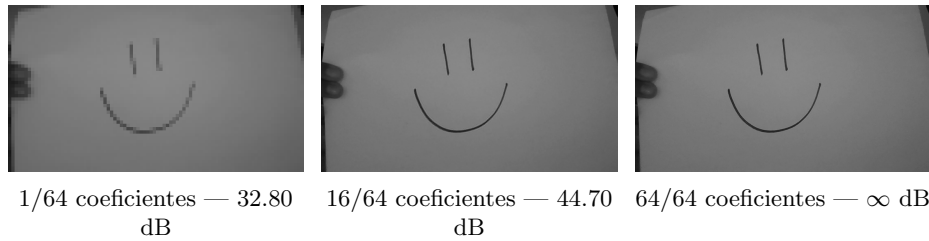
Se realizaron experimentos de compresión DCT sobre cuatro imágenes de prueba: dos capturadas directamente con la cámara del dispositivo a través de la aplicación (**cam1**: perro con textura densa de pelo, y **cam2**: dibujo sencillo sobre papel) y dos imágenes estándar de referencia (**lena** y **baboon**), ampliamente utilizadas en la literatura de procesamiento de imágenes [gonzalez'woods]. A cada imagen se le aplicaron tres configuraciones de coeficientes DCT conservados por bloque de 8×8 píxeles: 1/64, 16/64 y 64/64. La calidad de la reconstrucción se evaluó mediante la métrica PSNR (*Peak Signal-to-Noise Ratio*), que compara la imagen comprimida con la imagen original en escala de grises. Los resultados se resumen en la Tabla 1.

Las Figuras 1, 2, 3 y 4 muestran las imágenes reconstruidas para cada configuración de coeficientes.

Los resultados evidencian tres comportamientos consistentes. Primero, la imagen **baboon**, con mayor densidad de texturas de alta frecuencia espacial, registró el PSNR más bajo con 1 coeficiente (19.70 dB, calidad baja), mientras

Cuadro 1: Resumen de resultados de compresión DCT para las cuatro imágenes de prueba.

Imagen	Dimensiones	Coeeficientes	PSNR (dB)	Calidad	Tam. original	Tam. comprimido	Reducción
cam1	720×405	1/64	29.53	Aceptable	102.7 KB	6.0 KB	−94 %
cam1	720×405	16/64	44.61	Excelente	102.7 KB	83.7 KB	−19 %
cam1	720×405	64/64	∞	Sin pérdida	102.7 KB	102.7 KB	+0 %
cam2	720×405	1/64	32.80	Buena	88.5 KB	4.3 KB	−95 %
cam2	720×405	16/64	44.70	Excelente	88.5 KB	66.6 KB	−25 %
cam2	720×405	64/64	∞	Sin pérdida	88.5 KB	88.5 KB	+0 %
lena	512×512	1/64	23.66	Aceptable	148.3 KB	7.4 KB	−95 %
lena	512×512	16/64	34.62	Buena	148.3 KB	119.6 KB	−19 %
lena	512×512	64/64	∞	Sin pérdida	148.3 KB	148.3 KB	+0 %
baboon	512×512	1/64	19.70	Baja calidad	200.6 KB	7.9 KB	−96 %
baboon	512×512	16/64	24.16	Aceptable	200.6 KB	161.9 KB	−19 %
baboon	512×512	64/64	∞	Sin pérdida	200.6 KB	200.6 KB	+0 %

Figura 1: Imagen **cam1** comprimida con 1, 16 y 64 coeficientes DCT.Figura 2: Imagen **cam2** comprimida con 1, 16 y 64 coeficientes DCT.Figura 3: Imagen **lena** comprimida con 1, 16 y 64 coeficientes DCT.

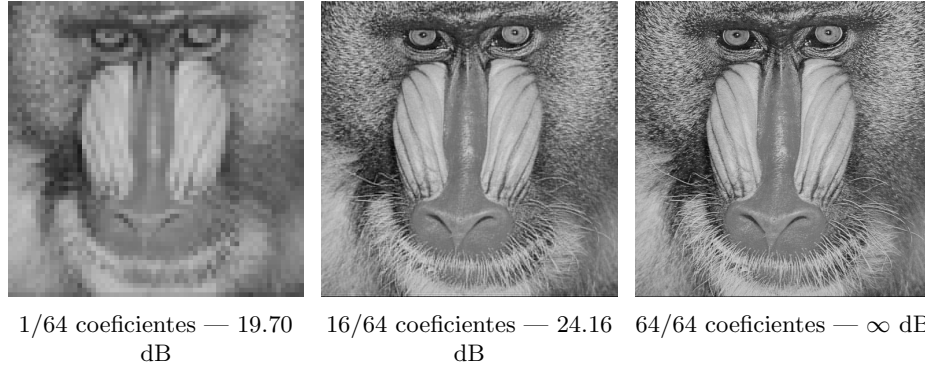


Figura 4: Imagen baboon comprimida con 1, 16 y 64 coeficientes DCT.

que **cam2**, cuya escena presenta grandes zonas uniformes (papel blanco con trazos simples), alcanzó 32.80 dB (calidad buena) con la misma configuración. Esto confirma que la DCT aprovecha la redundancia espacial: imágenes con pocas variaciones retienen mayor información con menos coeficientes. Segundo, al aumentar a 16 coeficientes, **cam1** y **cam2** alcanzaron calidad excelente (44.61 dB y 44.70 dB respectivamente), mientras que **baboon** permaneció en calidad aceptable (24.16 dB), lo que indica que las imágenes con alto contenido de detalle fino requieren un número mayor de coeficientes para una reconstrucción fiel. Tercero, con los 64 coeficientes del bloque se obtuvo reconstrucción sin pérdida ($\text{PSNR} = \infty$) en todas las imágenes, verificando que la transformada DCT-II aplicada bloque a bloque es reversible cuando se conservan todos los coeficientes.

4. Conclusiones

Los resultados obtenidos con las cuatro imágenes de prueba son consistentes con el comportamiento esperado: a medida que el número de coeficientes k aumenta, el PSNR crece y la calidad visual de la imagen reconstruida mejora, aproximándose progresivamente a la imagen original. Esto sugiere que el PSNR, si bien es una métrica práctica, no captura plenamente la complejidad del sistema visual humano.

Se comprobó que la compactación de energía de la DCT es efectiva: con tan solo 8 de los 64 coeficientes disponibles por bloque el sistema logra reconstruir una imagen reconocible, lo que evidencia que la mayor parte de la información visual de relevancia se concentra en las frecuencias bajas. Al conservar todos se recupera la imagen original sin pérdida, confirmando la reversibilidad de la transformada.

El recorrido zigzag comprobó ser una estrategia eficiente para la priorización de coeficientes, garantizando que los de mayor energía sean siempre seleccionados

primero.

La compresión DCT permite reducir considerablemente el tamaño de una imagen descartando coeficientes de alta frecuencia, y su eficacia depende de qué tan compleja sea la escena: mientras menos detalles finos tenga, menos coeficientes se necesitan para una reconstrucción aceptable. Por lo tanto, se debe buscar un buen equilibrio entre calidad y compresión, en base a los coeficientes que se retengan.

Referencias

- [1] Nasir Ahmed, T. Natarajan y Kamisetty R. Rao. “Discrete Cosine Transform”. En: *IEEE Transactions on Computers* C-23.1 (1974), págs. 90-93.
- [2] Rafael C. Gonzalez y Richard E. Woods. *Digital Image Processing*. Fourth. Pearson, 2018.
- [3] Alain Horé y Djemel Ziou. “Image Quality Metrics: PSNR vs. SSIM”. En: *Proceedings of the 20th International Conference on Pattern Recognition (ICPR)* (2010), págs. 2366-2369.
- [4] Kamisetty R. Rao y P. Yip. *Discrete Cosine Transform: Algorithms, Advantages, Applications*. Academic Press, 1990.

Anexos

```
import base64
import io
import math
import os

import numpy as np
from flask import Flask, jsonify, render_template, request
from PIL import Image
from scipy.fftpack import dct, idct

# Inicializacin de la aplicacin Flask
app = Flask(__name__)
# Lmite mximo de tamao de archivo subido: 16 MB
app.config["MAX_CONTENT_LENGTH"] = 16 * 1024 * 1024

# Dimensin mxima permitida (ancho o alto) antes de redimensionar
MAX_DIM = 720

def _zigzag_indices(n: int = 8):
    """
    Genera el orden de recorrido en zigzag para un bloque nn.
```

```

Este orden se usa para seleccionar los coeficientes DCT ms
importantes
(bajas frecuencias primero) al comprimir cada bloque.
"""
indices = []
r, c = 0, 0
going_up = True # Direccin actual del recorrido

for _ in range(n * n):
    indices.append((r, c)) # Registra la posicin actual

    if going_up:
        # Llegamos al borde superior: avanzar a la derecha y bajar
        if r == 0 and c < n - 1:
            c += 1
            going_up = False
        # Llegamos al borde derecho: bajar una fila
        elif c == n - 1:
            r += 1
            going_up = False
        else:
            # Movimiento diagonal hacia arriba-derecha
            r -= 1
            c += 1
    else:
        # Llegamos al borde izquierdo: avanzar hacia abajo
        if c == 0 and r < n - 1:
            r += 1
            going_up = True
        # Llegamos al borde inferior: avanzar a la derecha
        elif r == n - 1:
            c += 1
            going_up = True
        else:
            # Movimiento diagonal hacia abajo-izquierda
            r += 1
            c -= 1

    return indices

# Tabla de ndices zigzag precalculada para bloques 8x8
ZZ = _zigzag_indices()

def _resize(img: Image.Image) -> Image.Image:
    """
    Redimensiona la imagen si supera MAX_DIM en alguno de sus lados,
    manteniendo la proporcin original.
    """

```

```

w, h = img.size
if w > MAX_DIM or h > MAX_DIM:
    # Calcula el factor de escala para que ningn lado supere
    MAX_DIM
    scale = min(MAX_DIM / w, MAX_DIM / h)
    img = img.resize((round(w * scale), round(h * scale)), Image.
    LANCZOS)
return img

def compress_dct(img_gray: np.ndarray, num_coef: int) -> np.ndarray:
    """
    Comprime una imagen en escala de grises usando la Transformada
    Discreta
    del Coseno (DCT) por bloques de 88.

    Por cada bloque se aplica la DCT 2D, se conservan solo los
    primeros
    num_coef coeficientes segn el orden zigzag (bajas frecuencias), y
    se
    reconstruye el bloque con la IDCT 2D.
    """
    h, w = img_gray.shape
    data = img_gray.astype(np.float64) # Convertir a float64 para
    precisin numrica

    # Calcular dimensiones con padding para que sean mltiplos de 8
    ph = int(np.ceil(h / 8) * 8)
    pw = int(np.ceil(w / 8) * 8)

    # Rellenar los bordes replicando el pxel del borde (evita
    artefactos)
    padded = np.pad(data, ((0, ph - h), (0, pw - w)), mode="edge")
    result = np.zeros_like(padded) # Imagen resultado del mismo tamao
    que el padding

    # Recorrer la imagen en bloques de 88
    for bi in range(0, ph, 8):
        for bj in range(0, pw, 8):
            block = padded[bi : bi + 8, bj : bj + 8] # Extraer bloque
            actual

            # Aplicar DCT 2D separable: primero por filas, luego por
            columnas
            coefs = dct(dct(block, norm="ortho", axis=0), norm="ortho"
            , axis=1)

            # Crear mscara vaca y copiar solo los num_coef primeros
            coeficientes zigzag
            mask = np.zeros((8, 8), dtype=np.float64)

```

```

        for k in range(num_coef):
            r, c = ZZ[k]          # Posicin (fila, col) segn
orden zigzag
            mask[r, c] = coefs[r, c] # Conservar coeficiente

            # Reconstruir el bloque aplicando la IDCT 2D inversa
            recon = idct(idct(mask, norm="ortho", axis=1), norm="ortho
", axis=0)
            result[bi : bi + 8, bj : bj + 8] = recon # Guardar bloque
reconstruido

# Recortar el padding y limitar valores al rango vlido [0, 255]
cropped = result[:h, :w]
return np.clip(np.round(cropped), 0, 255).astype(np.uint8)

def psnr(original: np.ndarray, reconstructed: np.ndarray, bits: int =
8) -> float:
    """
    Calcula el PSNR (Peak Signal-to-Noise Ratio) entre la imagen
original
y la reconstruida, usando la frmula:

        PSNR = 10 * log10( (2^n - 1) / MSE )

    donde n es la profundidad de bits (por defecto 8) y MSE es el
error
cuadrático medio entre ambas imgenes.
    """
    # Error cuadrático medio entre la imagen original y la comprimida
    mse = float(
        np.mean((original.astype(np.float64) - reconstructed.astype(np
.float64)) ** 2)
    )

    # Si el MSE es prcticamente cero, las imgenes son idnticas    PSNR
infinito
    if mse < 1e-10:
        return float("inf")

    # Valor mximo posible al cuadrado segn la profundidad de bits
    max_val = (2 ** bits - 1) ** 2
    return float(10 * np.log10(max_val / mse))

def _to_b64_png(arr: np.ndarray) -> str:
    """
    Convierte un array NumPy (imagen) a una cadena PNG codificada en
Base64,
    lista para ser incrustada en una respuesta JSON.

```

```
"""
buf = io.BytesIO()                                # Buffer en
memoria
Image.fromarray(arr).save(buf, format="PNG")      # Guardar como PNG
en el buffer
return base64.b64encode(buf.getvalue()).decode("utf-8") #
Codificar en Base64

@app.route("/")
def index():
    """Sirve la pagina principal de la aplicacin."""
    return render_template("index.html")

@app.route("/process", methods=["POST"])
def process():
    """
    Recibe una imagen y el nmero de coeficientes DCT, aplica la
    compresin
    y devuelve en JSON las imgenes (original, grises, comprimida)
    junto con
    mtricas: PSNR, dimensiones y tamaos de archivo.
    """
    # Verificar que se envi un archivo de imagen
    if "image" not in request.files:
        return jsonify({"error": "No se recibí ninguna imagen."}), 400

    # Leer y validar el nmero de coeficientes DCT
    try:
        num_coef = int(request.form.get("num_coef", 8))
    except (TypeError, ValueError):
        return jsonify({"error": "num_coef debe ser un entero."}), 400

    if not (1 <= num_coef <= 64):
        return jsonify({"error": "num_coef debe estar entre 1 y 64."})
    , 400

    # Abrir la imagen y convertirla a RGB
    try:
        img_rgb = Image.open(request.files["image"].stream).convert("
        RGB")
    except Exception:
        return jsonify({"error": "No se pudo leer la imagen. Formato
        no soportado."}), 400

    # Redimensionar si supera el tamaño máximo permitido
    img_rgb = _resize(img_rgb)
    width, height = img_rgb.size
```

```

# Convertir a escala de grises para aplicar la DCT
img_gray = np.array(img_rgb.convert("L"))

# Aplicar la compresin DCT con el nmero de coeficientes indicado
img_comp = compress_dct(img_gray, num_coef)

# Calcular el PSNR entre la imagen original en grises y la
comprimida
p = psnr(img_gray, img_comp)

# Medir el tamao del PNG de la imagen en grises (antes de
comprimir)
buf_orig = io.BytesIO()
Image.fromarray(img_gray).save(buf_orig, format="PNG")
size_original = buf_orig.tell() # Tamao en bytes

# Medir el tamao del PNG de la imagen comprimida
buf_comp = io.BytesIO()
Image.fromarray(img_comp).save(buf_comp, format="PNG")
size_comprimida = buf_comp.tell() # Tamao en bytes

# Devolver todos los resultados como JSON
return jsonify(
    {
        "imagen_original": _to_b64_png(np.array(img_rgb)), #
        Imagen RGB original
        "imagen_referencia": _to_b64_png(img_gray), #
        Imagen en grises
        "imagen_comprimida": _to_b64_png(img_comp), #
        Imagen comprimida
        "psnr": round(p, 2) if math.isfinite(p) else
        None,
        "psnr_inf": not math.isfinite(p), #
        True si PSNR es infinito
        "num_coef": num_coef, #
        Coeficientes usados
        "width": width, #
        Ancho en pxeles
        "height": height, #
        Alto en pxeles
        "tasa_compresion": round(num_coef / 64, 4), #
        Fraccin de coeficientes conservados
        "size_original": size_original, #
        Bytes antes de comprimir
        "size_comprimida": size_comprimida, #
        Bytes despues de comprimir
    }
)

```

```
if __name__ == "__main__":  
    # Usar 8000  
    port = int(os.environ.get("PORT", 8000))  
    app.run(host="0.0.0.0", port=port, debug=False)
```